

JavaScript基础

什么是 DOM 和 BOM?

1、DOM 指的是文档对象模型，它指的是把文档当做一个对象，这个对象主要定义了处理网页内容的方法和接口。

2、BOM 指的是浏览器对象模型，它指的是把浏览器当做一个对象来对待，这个对象主要定义了与浏览器进行交互的方法和接口。BOM 的核心是 window，而 window 对象具有双重角色，它既是通过 js 访问浏览器窗口的一个接口，又是一个 Global（全局）对象。这意味着在网页中定义的任何对象，变量和函数，都作为全局对象的一个属性或者方法存在。

(1)location 对象

- location.href -- 返回或设置当前文档的 URL
- location.search -- 返回 URL 中的查询字符串部分。例如 <http://www.baidu.com/index.php?id=5&name=index> 返回包括(?)后面的内容?id=5&name=index
- location.hash -- 返回 URL #后面的内容，如果没有#，返回空
- location.host -- 返回 URL 中的域名部分，例如 www.baidu.com
- location.hostname -- 返回 URL 中的主域名部分，例如 baidu.com
- location.pathname -- 返回 URL 的域名后的部分。例如 <http://www.baidu.com/xhtml/> 返回/xhtml/
- location.port -- 返回 URL 中的端口部分。例如 <http://www.baidu.com:8080/xhtml/> 返回 8080
- location.protocol -- 返回 URL 中的协议部分。例如 <http://www.baidu.com:8080/xhtml/> 返回(//)前面的内容 http:
- location.assign -- 设置当前文档的 URL
- location.replace() -- 设置当前文档的 URL，并且在 history 对象的地址列表中移除这个
- URL location.replace(url);
- location.reload() -- 重载当前页面

(2)history 对象

- history.go() -- 前进或后退指定的页面数 history.go(num);
- history.back() -- 后退一页
- history.forward() -- 前进一页

(3)Navigator 对象

- navigator.userAgent -- 返回用户代理头的字符串表示(就是包括浏览器版本信息等的字符串)
- navigator.cookieEnabled -- 返回浏览器是否支持(启用)cookie

js 中 var 的变量和 function 的函数名重名时的执行结果

```
console.log(a);
var a = 100;
function a () {};
console.log(a);

console.log(a);
function a () {};
var a = 100;
console.log(a);
```

答案是两个场景输入都是一样的.结果都为:

```
f a() {}  
100
```

结论: 函数声明会覆盖变量声明

JS 为什么放到后面, CSS 会阻塞渲染吗

为什么外链 css 为什么要放头部?

首先整个页面展示给用户会经过 html 的解析与渲染过程。

而外链 css 无论放在 html 的任何位置都不影响 html 的解析, 但是影响 html 的渲染。

如果将 css 放在尾部, html 的内容可以第一时间显示出来, 但是会阻塞 html 行内 css 的渲染。

浏览器的这个策略其实很明智的, 想象一下, 如果没有这个策略, 页面首先会呈现出一个行内 css 样式, 待 CSS 下载完之后又突然变了一个模样。用户体验可谓极差, 而且渲染是有成本的。

如果将 css 放在头部, css 的下载解析是可以和 html 的解析同步进行的, 放到尾部, 要花费额外时间来解析 CSS, 并且浏览器会先渲染出一个没有样式的页面, 等 CSS 加载完后会再渲染成一个有样式的页面, 页面会出现明显的闪动的现象。

为什么 script 要放在尾部?

因为当浏览器解析到 script 的时候, 就会立即下载执行, 中断 html 的解析过程, 如果外部脚本加载时间很长 (比如一直无法完成下载), 就会造成网页长时间失去响应, 浏览器就会呈现“假死”状态, 这被称为“阻塞效应”。

具体的流程是这样的:

- 浏览器一边下载 HTML 网页, 一边开始解析。
- 解析过程中, 发现 script 标签
- 暂停解析, 网页渲染的控制权转交给 JavaScript 引擎
- 如果 script 标签引用了外部脚本, 就下载该脚本, 否则就直接执行
- 执行完毕, 控制权交还渲染引擎, 恢复往下解析 HTML 网页

JavaScript 脚本延迟加载的方式有哪些?

- 延迟加载就是等页面加载完成之后再加载 JavaScript 文件。js 延迟加载有助于提高页面加载速度。

一般有以下几种方式:

- **defer 属性:** 给 js 脚本添加 defer 属性, 这个属性会让脚本的加载与文档的解析同步解析, 然后在文档解析完成后再执行这个脚本文件, 这样的话就能使页面的渲染不被阻塞。多个设置了 defer 属性的脚本按规范来说最后是顺序执行的, 但是在一些浏览器中可能不是这样。
- **async 属性:** 给 js 脚本添加 async 属性, 这个属性会使脚本异步加载, 不会阻塞页面的解析过程, 但是当脚本加载完成后立即执行 js 脚本, 这个时候如果文档没有解析完成的话同样会阻塞。多个 async 属性的脚本的执行顺序是不可预测的, 一般不会按照代码的顺序依次执行。
- **动态创建 DOM 方式:** 动态创建 DOM 标签的方式, 可以对文档的加载事件进行监听, 当文档加载完成后再动态的创建 script 标签来引入 js 脚本。
- **使用 setTimeout 延迟方法:** 设置一个定时器来延迟加载 js 脚本文件
- **让 JS 最后加载:** 将 js 脚本放在文档的底部, 来使 js 脚本尽可能的在后来加载执行。

为什么函数的 arguments 参数是类数组而不是数组? 如何遍历类数组?

arguments 是一个对象, 它的属性是从 0 开始依次递增的数字, 还有 callee 和 length 等属性, 与数组相似; 但是它却没有数组常见的方法属性, 如 forEach, reduce 等, 所以叫它们类数组。

要遍历类数组, 有三个方法:

- (1) 将数组的方法应用到类数组上, 这时候就可以使用 call 和 apply 方法, 如:

```
function foo(){
  Array.prototype.forEach.call(arguments, a => console.log(a))
}
```

(2) 使用Array.from方法将类数组转化成数组:

```
function foo(){
  const arrArgs = Array.from(arguments)
  arrArgs.forEach(a => console.log(a))
}
```

(3) 使用展开运算符将类数组转化成数组

```
function foo(){
  const arrArgs = [...arguments]
  arrArgs.forEach(a => console.log(a))
}
```

对象属性的循环遍历

- for ... in
 - for in 循环会遍历对象自身和继承的可枚举属性，不包含 Symbol 属性。
 - 如果只需要对象自身的属性，可以通过 Object.prototype.hasOwnProperty() 进行过滤。
- Object.keys(), Object.values(), Object.entries()
 - ES5 引入了 Object.keys 方法，返回一个数组，成员是参数对象自身的（不含继承的）所有可遍历（enumerable）属性（不含 Symbol 属性）的键名。
- Object.getOwnPropertyNames()
 - Object.getOwnPropertyNames 返回一个数组，包含对象自身的所有属性（不含 Symbol 属性，但是包括不可枚举属性）的键名。
- Object.getOwnPropertySymbols()
 - Object.getOwnPropertySymbols 返回一个数组，包含对象自身的所有 Symbol 属性的键名。
- Reflect.ownKeys()
 - Reflect.ownKeys 返回一个数组，包含对象自身的（不含继承的）所有键名，不管键名是 Symbol 或字符串，也不管是否可枚举。

set()和 map()区别

- Map 对象保存键值对。任何值(对象或者原始值)都可以作为一个键或一个值。构造函数 Map 可以接受一个数组作为参数。
- Set 对象允许你存储任何类型的值，无论是原始值或者是对象引用。它类似于数组，但是成员的值都是唯一的，没有重复的值。

forEach和map方法有什么区别

这方法都是用来遍历数组的，两者区别如下：

- `forEach()`方法会针对每一个元素执行提供的函数，对数据的操作会改变原数组，该方法没有返回值；
- `map()`方法不会改变原数组的值，返回一个新数组，新数组中的值为原数组调用函数处理之后的值；

map和Object的区别

-	Map	Object
意外的键	Map默认情况不包含任何键，只包含显式插入的键。	Object 有一个原型, 原型链上的键名有可能和自己在对象上的设置的键名产生冲突。
键的类型	Map的键可以是任意值，包括函数、对象或任意基本类型。	Object 的键必须是 String 或是Symbol。
键的顺序	Map 中的 key 是有序的。因此，当迭代的时候， Map 对象以插入的顺序返回键值。	Object 的键是无序的
Size	Map 的键值对个数可以轻易地通过size 属性获取	Object 的键值对个数只能手动计算
迭代	Map 是 iterable 的，所以可以直接被迭代。	迭代Object需要以某种方式获取它的键然后才能迭代。
性能	在频繁增删键值对的场景下表现更好。	在频繁添加和删除键值对的场景下未作出优化。

(其他) JavaScript有哪些内置对象

全局的对象（ global objects ）或称标准内置对象，不要和 "全局对象（ global object ）" 混淆。这里说的全局的对象是说在 全局作用域里的对象。全局作用域中的其他对象可以由用户的脚本创建或由宿主程序提供。

标准内置对象的分类：

- （1）值属性，这些全局属性返回一个简单值，这些值没有自己的属性和方法。例如 Infinity、NaN、undefined、null 字面量
- （2）函数属性，全局函数可以直接调用，不需要在调用时指定所属对象，执行结束后会将结果直接返回给调用者。例如 `eval()`、`parseFloat()`、`parseInt()` 等
- （3）基本对象，基本对象是定义或使用其他对象的基础。基本对象包括一般对象、函数对象和错误对象。例如 Object、Function、Boolean、Symbol、Error 等
- （4）数字和日期对象，用来表示数字、日期和执行数学计算的对象。例如 Number、Math、Date
- （5）字符串，用来表示和操作字符串的对象。例如 String、RegExp
- （6）可索引的集合对象，这些对象表示按照索引值来排序的数据集合，包括数组和类型数组，以及类数组结构的对象。例如 Array

(7) 使用键的集合对象，这些集合对象在存储数据时会使用到键，支持按照插入顺序来迭代元素。例如 Map、Set、WeakMap、WeakSet

(8) 矢量集合，SIMD 矢量集合中的数据会被组织为一个数据序列。例如 SIMD 等

(9) 结构化数据，这些对象用来表示和操作结构化的缓冲区数据，或使用 JSON 编码的数据。例如 JSON 等

(10) 控制抽象对象 例如 Promise、Generator 等

(11) 反射。例如 Reflect、Proxy

(12) 国际化，为了支持多语言处理而加入 ECMAScript 的对象。例如 Intl、Intl.Collator 等

(13) WebAssembly

(14) 其他。例如 arguments

总结：js 中的内置对象主要指的是在程序执行前存在全局作用域里的由 js 定义的一些全局值属性、函数和用来实例化其他对象的构造函数对象。一般经常用到的如全局变量值 NaN、undefined，全局函数如 parseInt()、parseFloat() 用来实例化对象的构造函数如 Date、Object 等，还有提供数学计算的单体内置对象如 Math 对象。

(其他) use strict是什么意思？使用它区别是什么？

use strict 是一种 ECMAScript5 添加的（严格模式）运行模式，这种模式使得 Javascript 在更严格的条件下运行。设立严格模式的目的如下：

- 消除 Javascript 语法的不合理、不严谨之处，减少怪异行为；
- 消除代码运行的不安全之处，保证代码运行的安全；
- 提高编译器效率，增加运行速度；
- 为未来新版本的 Javascript 做好铺垫。

区别：

- 禁止使用 with 语句。
- 禁止 this 关键字指向全局对象。
- 对象不能有重名的属性。

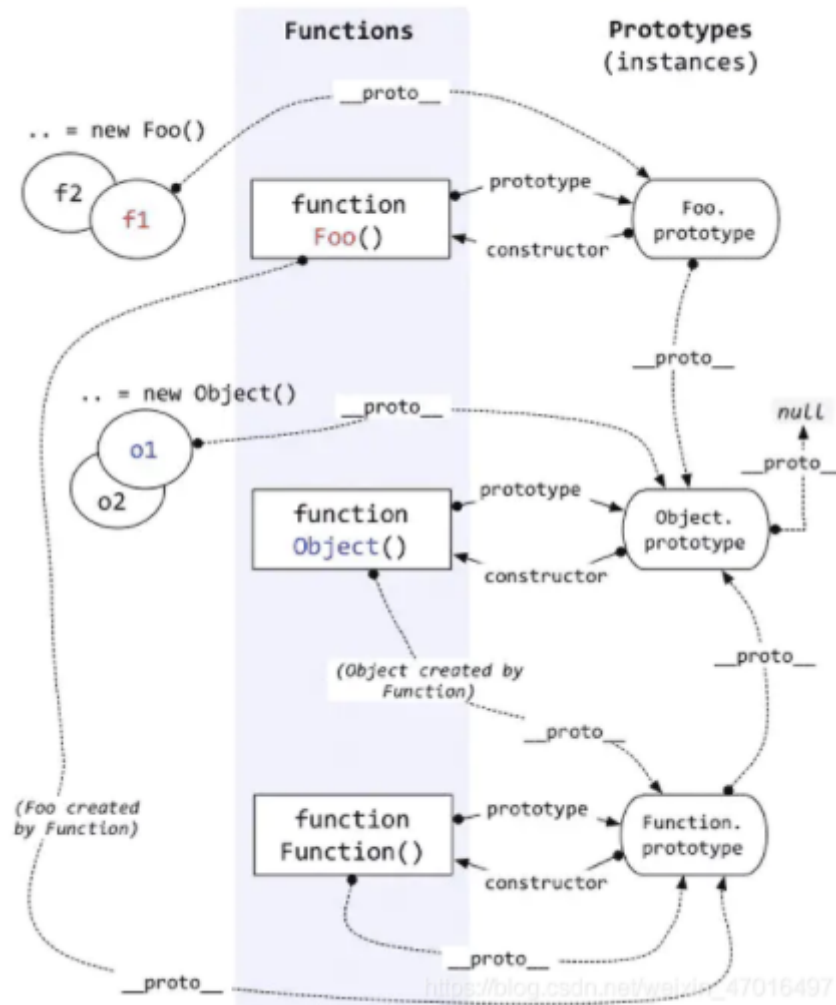
(其他) 原型与原型链

对原型、原型链的理解

在JavaScript中是使用构造函数来新建一个对象的，每一个构造函数的内部都有一个 prototype 属性，它的属性值是一个对象，这个对象包含了可以由该构造函数的所有实例共享的属性和方法。当使用构造函数新建一个对象后，在这个对象的内部将包含一个指针，这个指针指向构造函数的 prototype 属性对应的值，在 ES5 中这个指针被称为对象的原型。一般来说不应该能够获取到这个值的，但是现在浏览器中都实现了 **proto** 属性来访问这个属性，但是最好不要使用这个属性，因为它不是规范中规定的。ES5 中新增了一个 Object.getPrototypeOf() 方法，可以通过这个方法来获取对象的原型。

当访问一个对象的属性时，如果这个对象内部不存在这个属性，那么它就会去它的原型对象里找这个属性，这个原型对象又会有自己的原型，于是就这样一直找下去，也就是原型链的概念。原型链的尽头一般来说都是 Object.prototype 所以这就是新建的对象为什么能够使用 toString() 等方法的原因。

特点：JavaScript 对象是通过引用来传递的，创建的每个新对象实体中并没有一份属于自己的原型副本。当修改原型时，与之相关的对象也会继承这一改变。



原型修改、重写

```
function Person(name) {
  this.name = name
}
// 修改原型
Person.prototype.getName = function() {}
var p = new Person('hello')
console.log(p.__proto__ === Person.prototype) // true
console.log(p.__proto__ === p.constructor.prototype) // true
// 重写原型
Person.prototype = {
  getName: function() {}
}
var p = new Person('hello')
console.log(p.__proto__ === Person.prototype) // true
console.log(p.__proto__ === p.constructor.prototype) // false
```

可以看到修改原型的时候`p`的构造函数不是指向`Person`了，因为直接给`Person`的原型对象直接用对象赋值时，它的构造函数指向的了根构造函数`Object`，所以这时候 `p.constructor === Object`，而不是 `p.constructor === Person`。要想成立，就要用`constructor`指回来：

```
Person.prototype = {
  getName: function() {}
}
var p = new Person('hello')
p.constructor = Person
console.log(p.__proto__ === Person.prototype) // true
console.log(p.__proto__ === p.constructor.prototype) // true
```

原型链指向

```
p.__proto__ // Person.prototype
Person.prototype.__proto__ // Object.prototype
p.__proto__.__proto__ //Object.prototype
p.__proto__.constructor.prototype.__proto__ // Object.prototype
Person.prototype.constructor.prototype.__proto__ // Object.prototype
p1.__proto__.constructor // Person
Person.prototype.constructor // Person
```

原型链的终点是什么

由于 `Object` 是构造函数，原型链终点是 `Object.prototype.__proto__`，而 `Object.prototype.__proto__=== null // true`，所以，原型链的终点是 `null`。原型链上的所有原型都是对象，所有的对象最终都是由 `Object` 构造的，而 `Object.prototype` 的下一级是 `Object.prototype.__proto__`。

获得对象非原型链上的属性

使用后 `hasOwnProperty()` 方法来判断属性是否属于原型链的属性：

```
function iterate(obj){
  var res=[];
  for(var key in obj){
    if(obj.hasOwnProperty(key))
      res.push(key+' : '+obj[key]);
  }
  return res;
}
```

this关键字

- `this` 是执行上下文中的一个属性，它指向最后一次调用这个方法的对象。在实际开发中，`this` 的指向可以通过四种调用模式来判断。

第一种是**函数调用模式**，当一个函数不是一个对象的属性时，直接作为函数来调用时，`this` 指向全局对象。

- 全局上下文

```
console.log(window === this); // true
var a = 1;
this.b = 2;
window.c = 3;
console.log(a + b + c); // 6
```

- 函数上下文

```
function foo(){
  return this;
}
console.log(foo() === window); // true
```

- 箭头函数

```
function Person(name){
  this.name = name;
  this.say = () => {
    var name = "xb";
    return this.name;
  }
}
var person = new Person("axuebin");
console.log(person.say()); // axuebin
```

第二种是**方法调用模式**，如果一个函数作为一个对象的方法来调用时，this 指向这个对象。

```
var person = {
  name: "axuebin",
  getName: function(){
    return this.name;
  }
}
console.log(person.getName()); // axuebin
```

这里有一个需要注意的地方

```
var name = "xb";
var person = {
  name: "axuebin",
  getName: function(){
    return this.name;
  }
}
var getName = person.getName;
console.log(getName()); // xb
```

第三种是**构造器调用模式**，如果一个函数用 new 调用时，函数执行前会新创建一个对象，this 指向这个新创建的对象。

第四种是**apply、call 和 bind 调用模式**，这三个方法都可以显示的指定调用函数的 this 指向。其中 apply 方法接收两个参数：一个是 this 绑定的对象，一个是参数数组。call 方法接收的参数，第一个是 this 绑定的对象，后面的其余参数是传入函数执行的参数。也就是说，在使用 call() 方法时，传递给函数的参数必须逐个列举出来。bind 方法通过传入一个对象，返回一个 this 绑定了传入对象的新函数。这个函数的 this 指向除了使用 new 时会被改变，其他情况下都不会改变。


```
var person = {
    name: "axuebin",
    age: 25
};
function say(job) {
    console.log(this.name + ":" + this.age + " " + job);
}
say.call(person, "FE"); // axuebin:25 FE
say.apply(person, ["FE"]); // axuebin:25 FE
var sayPerson = say.bind(person, "FE");
sayPerson(); // axuebin:25 FE
```

这四种方式，使用构造器调用模式的优先级最高，然后是 apply、call 和 bind 调用模式，然后是方法调用模式，然后是函数调用模式。

作用域链/闭包

闭包

闭包是指有权访问另一个函数作用域中变量的函数，创建闭包的最常见的方式就是在一个函数内创建另一个函数，创建的函数可以访问到当前函数的局部变量。

闭包有两个常用的用途：

- 闭包的第一个用途是使我们在函数外部能够访问到函数内部的变量。通过使用闭包，可以通过在外部调用闭包函数，从而在外部访问到函数内部的变量，可以使用这种方法来创建私有变量。
- 闭包的另一个用途是使已经运行结束的函数上下文中的变量对象继续留在内存中，因为闭包函数保留了这个变量对象的引用，所以这个变量对象不会被回收。

比如，函数 A 内部有一个函数 B，函数 B 可以访问到函数 A 中的变量，那么函数 B 就是闭包。

```
function A() {
    let a = 1
    window.B = function () {
        console.log(a)
    }
}
A()
B() // 1
```

在 JS 中，闭包存在的意义就是让我们可以间接访问函数内部的变量。经典面试题：循环中使用闭包解决 var 定义函数的问题

```
for (var i = 1; i <= 5; i++) {
    setTimeout(function timer() {
        console.log(i)
    }, i * 1000)
}
```

首先因为 setTimeout 是个异步函数，所以会先把循环全部执行完毕，这时候 i 就是 6 了，所以会输出一堆 6。解决办法有三种：

- 第一种是使用闭包的方式

```
for (var i = 1; i <= 5; i++) {
;(function(j) {
    setTimeout(function timer() {
        console.log(j)
    }, j * 1000)
})(i)
}
```

在上述代码中，首先使用了立即执行函数将 `i` 传入函数内部，这个时候值就被固定在了参数 `j` 上面不会改变，当下次执行 `timer` 这个闭包的时候，就可以使用外部函数的变量 `j`，从而达到目的。

- 第二种就是使用 `setTimeout` 的第三个参数，这个参数会被当成 `timer` 函数的参数传入。

```
for (var i = 1; i <= 5; i++) {
    setTimeout(
        function timer(j) {
            console.log(j)
        },
        i * 1000,
        i
    )
}
```

- 第三种就是使用 `let` 定义 `i` 来解决问题了，这个也是最为推荐的方式

```
for (let i = 1; i <= 5; i++) {
    setTimeout(function timer() {
        console.log(i)
    }, i * 1000)
}
```

执行上下文介绍

- 简而言之，执行上下文是评估和执行 JavaScript 代码的环境的抽象概念。每当 Javascript 代码在运行的时候，它都是在执行上下文中运行。

执行上下文的类型

- 全局执行上下文 — 这是默认或者说基础的上下文，任何不在函数内部的代码都在全局上下文中。它会执行两件事：创建一个全局的 `window` 对象（浏览器的情况下），并且设置 `this` 的值等于这个全局对象。一个程序中只会有一个全局执行上下文。
- 函数执行上下文 — 每当一个函数被调用时，都会为该函数创建一个新的上下文。每个函数都有它自己的执行上下文，不过是在函数被调用时创建的。函数上下文可以有任意多个。每当一个新的执行上下文被创建，它会按定义的顺序（将在后文讨论）执行一系列步骤。
- `Eval` 函数执行上下文 — 执行在 `eval` 函数内部的代码也会有它属于自己的执行上下文，但由于 JavaScript 开发者并不经常使用 `eval`，所以在这里我不会讨论它。

执行栈

- 执行栈，也就是在其它编程语言中所说的“调用栈”，是一种拥有 LIFO（后进先出）数据结构的栈，被用来存储代码运行时创建的所有执行上下文。

怎么创建执行上下文？

- 创建执行上下文有两个阶段：1) 创建阶段 和 2) 执行阶段。
- 在创建阶段会发生三件事：

- this 值的决定，即我们所熟知的 This 绑定。
 - 创建词法环境组件。
 - 创建变量环境组件。
- 执行阶段
 - 在此阶段，完成对所有这些变量的分配，最后执行代码。

作用域链

- 当查找变量的时候，会先从当前上下文的变量对象中查找，如果没有找到，就会从父级(词法层面上的父级)执行上下文的变量对象中查找，一直找到全局上下文的变量对象，也就是全局对象。这样由多个执行上下文的变量对象构成的链表就叫做作用域链。

数据类型

JavaScript有哪些数据类型

JavaScript共有八种数据类型，分别是 **Undefined**、**Null**、**Boolean**、**Number**、**String**、**Object**、**Symbol**、**BigInt**。

其中 Symbol 和 BigInt 是ES6 中新增的数据类型：

- Symbol 代表创建后独一无二且不可变的数据类型，它主要是为了解决可能出现的全局变量冲突的问题。
- BigInt 是一种数字类型的数据，它可以表示任意精度格式的整数，使用 BigInt 可以安全地存储和操作大整数，即使这个数已经超出了 Number 能够表示的安全整数范围。

这些数据可以分为原始数据类型和引用数据类型：

- 栈：原始数据类型 (Undefined、Null、Boolean、Number、String)
- 堆：引用数据类型 (对象、数组和函数)

两种类型的区别在于存储位置的不同：

- 原始数据类型直接存储在栈 (stack) 中的简单数据段，占据空间小、大小固定，属于被频繁使用数据，所以放入栈中存储；
- 引用数据类型存储在堆 (heap) 中的对象，占据空间大、大小不固定。如果存储在栈中，将会影响程序运行的性能；引用数据类型在栈中存储了指针，该指针指向堆中该实体的起始地址。当解释器寻找引用值时，会首先检索其在栈中的地址，取得地址后从堆中获得实体。

堆和栈的概念存在于数据结构和操作系统内存中，在数据结构中：

- 在数据结构中，栈中数据的存取方式为先进后出。
- 堆是一个优先队列，是按优先级来进行排序的，优先级可以按照大小来规定。

在操作系统中，内存被分为栈区和堆区：

- 栈区内存由编译器自动分配释放，存放函数的参数值，局部变量的值等。其操作方式类似于数据结构中的栈。
- 堆区内存一般由开发着分配释放，若开发者不释放，程序结束时可能由垃圾回收机制回收。

JavaScript 判断数组的五种方法

instanceof

```
const arr= []
instanceof arr === Array // true
```

Array.isArray

```
const arr = []
Array.isArray(arr) // true

const obj = {}
Array.isArray(obj) // false
```

Object.prototype.isPrototypeOf

- 使用 Object 的原型方法 isPrototypeOf，判断两个对象的原型是否一样，isPrototypeOf() 方法用于测试一个对象是否存在于另一个对象的原型链上。

```
const arr = []
Object.prototype.isPrototypeOf(arr, Array.prototype) // true
```

Object.getPrototypeOf

```
const arr = []
Object.getPrototypeOf(arr) === Array.prototype // true
```

Object.prototype.toString

```
const arr = []
Object.prototype.toString.call(arr) === '[object Array]' // true

const obj = {}
Object.prototype.toString.call(obj) // "[object Object]"
```

- 为什么不直接用 obj.toString() 呢？
这是因为 toString 为 Object 的原型方法，而 Array，function 等类型作为 Object 的实例，都重写了 toString 方法。不同的对象类型调用 toString 方法时，根据原型链的知识，调用的是对应的重写之后的 toString 方法，而不会去调用 Object 上原型 toString 方法（返回对象的具体类型），所以采用 obj.toString() 不能得到其对象类型，只能将 obj 转换为字符串类型；因此，在想要得到对象的具体类型时，应该调用 Object 上原型 toString 方法。

null 和 undefined 的差异

- null 转为数字类型值为 0，而 undefined 转为数字类型为 NaN (Not a Number)
- undefined 是代表调用一个值而该值却没有赋值，这时候默认则为 undefined
- null 是一个很特殊的对象，最为常见的一个用法就是作为参数传入（说明该参数不是对象）
- 设置为 null 的变量或者对象会被内存收集器回收

(其他) typeof NaN 的结果是什么？

NaN 指“不是一个数字”（not a number），NaN 是一个“警戒值”（sentinel value，有特殊用途的常规值），用于指出数字类型中的错误情况，即“执行数学运算没有成功，这是失败后返回的结果”。

```
typeof NaN; // "number"
```

NaN 是一个特殊值，它和自身不相等，是唯一一个非自反（自反，reflexive，即 $x === x$ 不成立）的值。而 $NaN !== NaN$ 为 true。

(其他) isNaN 和 Number.isNaN 函数的区别

- 函数 isNaN 接收参数后，会尝试将这个参数转换为数值，任何不能被转换为数值的值都会返回 true，因此非数字值传入也会返回 true，会影响 NaN 的判断。
- 函数 Number.isNaN 会首先判断传入参数是否为数字，如果是数字再继续判断是否为 NaN，不会进行数据类型的转换，这种方法对于 NaN 的判断更为准确。

JavaScript 中 ==、=== 和 Object.is() 的区别

- ==: 等同，比较运算符，两边值类型不同的时候，先进行类型转换，再比较；
- ===: 恒等，严格比较运算符，不做类型转换，类型不同就是不等；
- Object.is() 是 ES6 新增的用来比较两个值是否严格相等的方法，与 === 的行为基本一致。
- 先说 ===，这个比较简单，只需要利用下面的规则来判断两个值是否恒等就行了：
 - 如果类型不同，就不相等
 - 如果两个都是数值，并且是同一个值，那么相等；
 - 值得注意的是，如果两个值中至少一个是 NaN，那么不相等（判断一个值是否是 NaN，可以用 isNaN() 或 Object.is() 来判断）。
 - 如果两个都是字符串，每个位置的字符都一样，那么相等；否则不相等。
 - 如果两个值都是同样的 Boolean 值，那么相等。
 - 如果两个值都引用同一个对象或函数，那么相等，即两个对象的物理地址也必须保持一致；否则不相等。
 - 如果两个值都是 null，或者都是 undefined，那么相等。
- 再说 Object.is()，其行为与 === 基本一致，不过有两处不同：
 - +0 不等于 -0。
 - NaN 等于自身。

isNaN 和 Number.isNaN 函数的区别？

- 函数 isNaN 接收参数后，会尝试将这个参数转换为数值，任何不能被转换为数值的值都会返回 true，因此非数字值传入也会返回 true，会影响 NaN 的判断。
- 函数 Number.isNaN 会首先判断传入参数是否为数字，如果是数字再继续判断是否为 NaN，不会进行数据类型的转换，这种方法对于 NaN 的判断更为准确。

JavaScript 中的包装类型

在 JavaScript 中，基本类型是没有属性和方法的，但是为了便于操作基本类型的值，在调用基本类型的属性或方法时 JavaScript 会在后台隐式地将基本类型的值转换为对象，如：

```
const a = "abc";
a.length; // 3
a.toUpperCase(); // "ABC"
```

在访问 'abc'.length 时，JavaScript 将 'abc' 在后台转换成 String('abc')，然后再访问其 length 属性。JavaScript 也可以使用 **Object** 函数显式地将基本类型转换为包装类型：

```
var a = 'abc'
Object(a) // String {"abc"}
```

也可以使用 **valueOf** 方法将包装类型倒转成基本类型：

```
var a = 'abc'
var b = Object(a)
var c = b.valueOf() // 'abc'
```

看看如下代码会打印出什么：

```
var a = new Boolean( false );
if (!a) {
    console.log( "oops" ); // never runs
}
```

答案是什么都不会打印，因为虽然包裹的基本类型是false，但是false被包裹成包装类型后就成了对象，所以其非值为false，所以循环体中的内容不会运行。

数组扁平化

- 数组扁平化就是把多维数组转化成一维数组。

flat(depth)

```
let a = [1,[2,3]];
a.flat(); // [1,2,3]
a.flat(1); //[1,2,3]
```

reduce

```
function flatten(arr){
    return arr.reduce(function(prev, cur){
        return prev.concat(Array.isArray(cur) ? flatten(cur) : cur)
    }, [])
}
const arr = [1, [2, [3, 4]]];
console.log(flatten(arr));
```

解构运算符 ...

```
function flatten(arr){
    while(arr.some(item => Array.isArray(item))){
        arr = [].concat(...arr);
    }
    return arr;
}

const arr = [1, [2, [3, 4]]];
console.log(flatten(arr));
```

数组去重

for 双重循环

```
function Array_dfor(data) {
    const newArray = [];
    let isRepeat;
    for (let i = 0; i < data.length; i++) {
```

```

    isRepeat = false;
    for (let j = 0; j < newArray.length; j++) {
        if (data[i] === newArray[j]) {
            isRepeat = true;
            break;
        }
    }
    if (!isRepeat) {
        newArray.push(data[i]);
    }
}
return newArray;
}

```

includes()

```

function Array_includes(data) {
    var arr = [];
    for (var i = 0; i < data.length; i++) {
        if (!arr.includes(data[i])) {
            arr.push(data[i])
        }
    }
    return arr;
}

```

indexOf()

```

function Array_indexOf(data) {
    var arr = [];
    for (var i = 0; i < data.length; i++) {
        if (arr.indexOf(data[i]) === -1){
            arr.push(data[i])
        }
    }
    return arr;
}

```

Map()

```

function Array_Map(data) {
    const newArr = [];
    const tmp = new Map();
    for (var i = 0; i < data.length; i++) {
        if (!tmp.has(data[i])){
            tmp.set(data[i],1)
            newArr.push(data[i])
        }
    }
    return newArr;
}

```

Set()

```
function Array_set(data) {  
  return Array.from(new Set(data))  
}
```

ES6

let、const、var的区别

(1) 块级作用域：块级作用域由 {} 包括，let 和 const 具有块级作用域，var 不存在块级作用域。块级作用域解决了 ES5 中的两个问题：

- 内层变量可能覆盖外层变量
- 用来计数的循环变量泄露为全局变量

(2) 变量提升：var 存在变量提升，let 和 const 不存在变量提升，即在变量只能在声明之后使用，否则会报错。

(3) 给全局添加属性：浏览器的全局对象是 window，Node 的全局对象是 global。var 声明的变量为全局变量，并且会将该变量添加为全局对象的属性，但是 let 和 const 不会。

(4) 重复声明：var 声明变量时，可以重复声明变量，后声明的同名变量会覆盖之前声明的遍历。const 和 let 不允许重复声明变量。

(5) 暂时性死区：在使用 let、const 命令声明变量之前，该变量都是不可用的。这在语法上，称为暂时性死区。使用 var 声明的变量不存在暂时性死区。

(6) 初始值设置：在变量声明时，var 和 let 可以不用设置初始值。而 const 声明变量必须设置初始值。

(7) 指针指向：let 和 const 都是 ES6 新增的用于创建变量的语法。let 创建的变量是可以更改指针指向（可以重新赋值）。但 const 声明的变量是不允许改变指针的指向。

区别	var	let	const
是否有块级作用域	×	✓	✓
是否存在变量提升	✓	×	×
是否添加全局属性	✓	×	×
能否重复声明变量	✓	×	×
是否存在暂时性死区	×	✓	✓
是否必须设置初始值	×	×	✓
能否改变指针指向	✓	✓	×

 牛客@AprilEcho

暂时性死区

- ES6 新增的 let、const 关键字声明的变量会产生块级作用域，如果变量在当前作用域中被创建之前被创建出来，由于此时还未完成语法绑定，如果我们访问或使用该变量，就会产生暂时性死区的问题，由此我们可以得知，从变量的创建到语法绑定之间这一段空间，我们就可以理解为‘暂时性死区’

箭头函数

- 箭头函数没有 arguments
- 箭头函数没有 prototype 属性，不能用作构造函数
- 箭头函数没有自己 this，它的 this 是词法的，引用的是上下文的 this，即在你写这行代码的时候就箭头函数的 this 就已经和外层执行上下文的 this 绑定了(这里个人认为并不代表完全是静态的,因为外层的上下文仍是动态的可以使用 call,apply,bind 修改,这里只是说明了箭头函数的 this 始终等于它上层上下文中的 this)
- 箭头函数的 this 指向即使使用 call,apply,bind 也无法改变（这里也验证了为什么 ECMAScript 规定不能使用箭头函数作为构造函数，因为它的 this 已经确定好了无法改变）

iterator 迭代器

- 可迭代的数据结构会有一个[Symbol.iterator]方法
- [Symbol.iterator]执行后返回一个 iterator 对象
- iterator 对象有一个 next 方法
- 执行一次 next 方法(消耗一次迭代器)会返回一个有 value,done 属性的对象

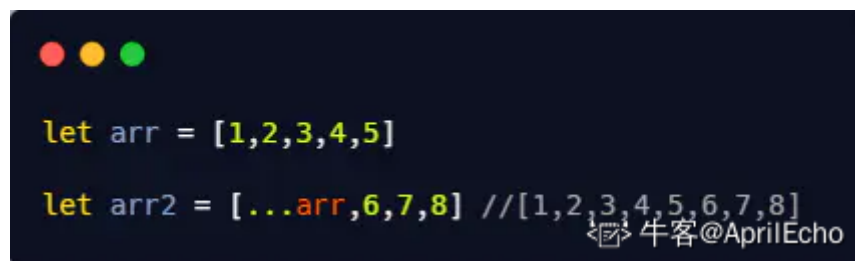
解构赋值

- 解构赋值可以直接使用对象的某个属性，而不需要通过属性访问的形式使用，对象解构原理个人认为通过寻找相同的属性名，然后原对象的这个属性名的值赋值给新对象对应的属性

剩余/扩展运算符

扩展运算符

- 以数组为例,使用扩展运算符使得可以"展开"这个数组，可以这么理解，数组是存放元素集合的一个容器，而使用扩展运算符可以将这个容器拆开，这样就只剩下元素集合，你可以把这些元素集合放到另外一个数组里面。



```
let arr = [1,2,3,4,5]
let arr2 = [...arr,6,7,8] // [1,2,3,4,5,6,7,8]
```

剩余运算符

- 剩余运算符最重要的一个特点就是替代了以前的 arguments

```
function func(a,b,c) {
    console.log(arguments[0],arguments[1],arguments[2])
}

func(1,2,3)

//rest是形参，承载了所有的函数参数，可以随意取名

function func1(...rest) {
    console.log(rest) // [1,2,3]
}

func1(1,2,3)
```

剩余运算符和扩展运算符的区别就是，剩余运算符会收集这些集合，放到右边的数组中，扩展运算符是将右边的数组拆分成元素的集合，它们是相反的

对象属性/方法简写

- 对象属性简写
- es6 允许当对象的属性和值相同时，省略属性名

```
let x = 0

let obj = {
    x //es6允许省略x:
}
```

需要注意的是 省略的是属性名而不是值 值必须是一个变量

- 方法简写

```
//es5
let obj = {
  func:function() {

  }
}

//es6方法的简写
let obj2 = {
  func() {

  }
}

//当然使用箭头函数也可以这么写
let obj3 = {
  func: () => {

  }
}
```

牛客@AprilEcho

for ... of 循环

- for ... of 是作为 ES6 新增的遍历方式,允许遍历一个含有 iterator 接口的数据结构并且返回各项的值
- ES3 中的 for ... in 的区别如下 :
 - 1、for ... of 只能用在可迭代对象上,获取的是迭代器返回的 value 值,for ... in 可以获取所有对象的键名
 - 2、for ... in 会遍历对象的整个原型链,性能非常差不推荐使用,而 for ... of 只遍历当前对象不会遍历它的原型链
 - 3、对于数组的遍历,for ... in 会返回数组中所有可枚举的属性(包括原型链上可枚举的属性),for ... of 只返回数组的下标对应的属性值

Promise (常用)

(详细见异步编程部分介绍)

Module(常用)

- ES6 Module 使用 import 关键字导入模块, export 关键字导出模块, 它还有以下特点
- ES6 Module 是静态的, 也就是说它是在编译阶段运行, 和 var 以及 function 一样具有提升效果 (这个特点使得它支持 tree shaking)
- 自动采用严格模式 (顶层的 this 返回 undefined)
- ES6 Module 支持使用 export {<变量>}导出具名的接口, 或者 export default 导出匿名的接口

函数默认值

- ES6 允许在函数的参数中设置默认值
ES5 写法:

```
function func(a) {  
    return a || 1  
}  
  
func() //1
```

ES6写法:

```
function func(a = 1) {  
    return a  
}  
  
func() //1
```

(其他) ES6模块与CommonJS模块有什么异同?

ES6 Module和CommonJS模块的区别:

- CommonJS是对模块的浅拷贝, ES6 Module是对模块的引用, 即ES6 Module只存只读, 不能改变其值, 也就是指针指向不能变, 类似const;
- import的接口是read-only (只读状态), 不能修改其变量值。即不能修改其变量的指针指向, 但可以改变变量内部指针指向, 可以对commonJS对重新赋值(改变指针指向), 但是对ES6 Module赋值会编译报错。

ES6 Module和CommonJS模块的共同点:

- CommonJS和ES6 Module都可以对引入的对象进行赋值, 即对对象内部属性的值进行改变。

ES7 的特性

Array.prototype.includes()

- includes() 函数用来判断一个数组是否包含一个指定的值, 如果包含则返回 true, 否则返回 false。
- includes 函数与 indexOf 函数很相似, 下面两个表达式是等价的:
arr.includes(x)arr.indexOf(x) >= 0
- 使用 indexOf()验证数组中是否存在某个元素, 这时需要根据返回值是否为-1 来判断:

```
let arr = ['react', 'angular', 'vue'];  
if (arr.indexOf('react') !== -1){  
    console.log('react 存在');  
}
```

指数操作符

- 在 ES7 中引入了指数运算符, 具有与 Math.pow(..)等效的计算结果。

```
console.log(2**10); // 输出 1024
```

ES8 的特性

- async/await
- Object.values()
- Object.entries()
- String padding
- 函数参数列表结尾允许逗号
- Object.getOwnPropertyDescriptors()

async/await

它也是为了解决回调地狱的问题，它只是一种语法糖。从本质上讲，await 函数仍然是 promise，其原理跟 Promise 相似。不过比起 Promise 之后用 then 方法来执行相关异步操作，async/await 则把异步操作变得更像传统函数操作。

- async 用于声明一个异步函数，该函数执行完之后返回一个 Promise 对象，可以使用 then 方法添加回调函数。

```
async function helloAsync(){
  return "helloAsync";
}
console.log(helloAsync()); // Promise {<resolved>: "helloAsync"}
helloAsync().then(v=>{
  console.log(v); // helloAsync
})
```

通过上面的代码可以得出结论，async 函数内 return 的值会被封装成一个 Promise 对象，由于 async 函数返回 Promise 对象，所以该函数可以按照 Promise 对象标准使用 then 方法进行后续异步操作。

（如果要把 async 函数方法跟 Promise 对象方法做对比的话，那么下面的 Promise 对象异步方法代码是完全相等于是上面的 async 函数异步方法。）

```
var helloAsync = function(){
  return new Promise(function(resolve){
    resolve("helloAsync");
  })
}
console.log(helloAsync())
helloAsync().then(v=>{
  console.log(v);
})
```

async 函数运行的时候是同步运行的，Promise 对象本身内容也是同步运行，这一点两者也是一致的，只有在 then 方法的时候才会被放入异步队列。

await

await 操作符用于等待一个 Promise 对象，它只能在异步函数 async function 内部使用。

async 函数运行的时候是同步运行，但是当 async 函数内部存在 await 操作符的时候，则会把 await 操作符标示的内容同步执行，await 操作符标示的内容之后的代码则被放入异步队列等待。

（await 标识的代码表示该代码运行需要一定的时间，所以后续的代码得进异步队列等待）

下面放一段 await 标准用法：

```
function testAwait (x) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

async function helloAsync() {
  var x = await testAwait ("hello world");
  console.log(x);
}

helloAsync ();
```

其实 await 多多少少对应了 Promise 对象异步方法里面的 then 方法，可以将上面代码改写成下面样式，结果也是一致的：

```
function testAwait (x) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

async function helloAsync() {
  var x = testAwait ("hello world");//此处 x 是一个 Promise 对象
  x.then(function(value){
    console.log(value);
  });
}

helloAsync ();
// hello world
```

上述方法把 await 去掉，使用 then 取代，能够起到同样的作用。两者都是把特定区域的代码放到异步队列中执行。

Object.values()

- Object.values()是一个与 Object.keys()类似的新函数，但返回的是 Object 自身属性的所有值，不包括继承的值。

```
const obj = {a: 1, b: 2, c: 3};
const values=Object.values(obj);
console.log(values);//[1, 2, 3]
```

Object.entries

- Object.entries()函数返回一个给定对象自身可枚举属性的键值对的数组。

String padding

- 在 ES8 中 String 新增了两个实例函数 String.prototype.padStart 和 String.prototype.padEnd，允许将空字符串或其他字符串添加到原始字符串的开头或结尾。

```
'x'.padStart(5, 'ab') // 'ababx'
'x'.padStart(4, 'ab') // 'abax'
'x'.padEnd(5, 'ab') // 'xabab'
'x'.padEnd(4, 'ab') // 'xaba'
```

异步编程

异步编程的实现方式

- 回调函数 的方式，使用回调函数的方式有一个缺点是，多个回调函数嵌套的时候会造成回调函数地狱，上下两层的回调函数间的代码耦合度太高，不利于代码的可维护。
- Promise 的方式，使用 Promise 的方式可以将嵌套的回调函数作为链式调用。但是使用这种方法，有时会造成多个 then 的链式调用，可能会造成代码的语义不够明确。
- generator 的方式，它可以在函数的执行过程中，将函数的执行权转移出去，在函数外部还可以将执行权转移回来。当遇到异步函数执行的时候，将函数执行权转移出去，当异步函数执行完毕时再将执行权给转移回来。因此在 generator 内部对于异步操作的方式，可以以同步的顺序来书写。使用这种方式需要考虑的问题是何时将函数的控制权转移回来，因此需要有一个自动执行 generator 的机制，比如说 co 模块等方式来实现 generator 的自动执行。
- async 函数 的方式，async 函数是 generator 和 promise 实现的一个自动执行的语法糖，它内部自带执行器，当函数内部执行到一个 await 语句的时候，如果语句返回一个 promise 对象，那么函数将会等待 promise 对象的状态变为 resolve 后再继续向下执行。因此可以将异步逻辑，转化为同步的顺序来书写，并且这个函数可以自动执行。

Promise

什么是 Promise

- Promise是异步编程的一种解决方案，它是一个对象，可以获取异步操作的消息，他的出现大大改善了异步编程的困境，避免了地狱回调，它比传统的解决方案回调函数和事件更合理和更强大。
- 所谓Promise，简单说就是一个容器，里面保存着某个未来才会结束的事件（通常是一个异步操作）的结果。从语法上说，Promise 是一个对象，从它可以获取异步操作的消息。Promise 提供统一的 API，各种异步操作都可以用同样的方法进行处理。
- Promise的实例有三个状态：
 - Pending（进行中）
 - Resolved（已完成）
 - Rejected（已拒绝）

当把一件事情交给promise时，它的状态就是Pending，任务完成了状态就变成了Resolved、没有完成失败了就变成了Rejected。

- Promise的实例有两个过程：
 - pending -> fulfilled : Resolved（已完成）
 - pending -> rejected : Rejected（已拒绝）
- Promise的特点：
 - 对象的状态不受外界影响。promise对象代表一个异步操作，有三种状态，pending（进行中）、fulfilled（已成功）、rejected（已失败）。只有异步操作的结果，可以决定当前是哪一种状态，任何其他操作都无法改变这个状态，这也是promise这个名字的由来——“承诺”；
 - 一旦状态改变就不会再变，任何时候都可以得到这个结果。promise对象的状态改变，只有两种可能：从pending变为fulfilled，从pending变为rejected。这时就称为resolved（已定型）。如果改变已经发生了，你再对promise对象添加回调函数，也会立即得到这个结果。这与事件（event）完全不同，事件的特点是：如果你错过了它，再去监听是得不到结果的。

Promise解决了什么问题

如题

```
let fs = require('fs')
fs.readFile('./a.txt', 'utf8', function(err, data){
  fs.readFile(data, 'utf8', function(err, data){
    fs.readFile(data, 'utf8', function(err, data){
      console.log(data)
    })
  })
})
```

上面的代码有如下缺点：

- 后一个请求需要依赖于前一个请求成功后，将数据往下传递，会导致多个ajax请求嵌套的情况，代码不够直观。
- 如果前后两个请求不需要传递参数的情况下，那么后一个请求也需要前一个请求成功后再执行下一步操作，这种情况下，那么也需要如上编写代码，导致代码不够直观。

使用promise方法对代码进行改进

```
let fs = require('fs')
function read(url){
  return new Promise((resolve, reject)=>{
    fs.readFile(url, 'utf8', function(error, data){
      error && reject(error)
      resolve(data)
    })
  })
}
read('./a.txt').then(data=>{
  return read(data)
}).then(data=>{
  return read(data)
}).then(data=>{
  console.log(data)
})
```

总结：promise可以解决了地狱回调的问题。

Promise方法

then()

- then方法可以接受两个回调函数作为参数。第一个回调函数是Promise对象的状态变为resolved时调用，第二个回调函数是Promise对象的状态变为rejected时调用。其中第二个参数可以省略。then方法返回的是一个新的Promise实例（不是原来那个Promise实例）。因此可以采用链式写法，即then方法后面再调用另一个then方法。

```
let promise = new Promise((resolve, reject)=>{
  ajax('first').success(function(res){
    resolve(res);
  })
})
promise.then(res=>{
  return new Promise((resovle, reject)=>{
```



```

        ajax('second').success(function(res){
            resolve(res)
        })
    })
}).then(res=>{
    return new Promise((resolve,reject)=>{
        ajax('second').success(function(res){
            resolve(res)
        })
    })
}).then(res=>{

})

```

catch()

- 该方法相当于then方法的第二个参数，指向reject的回调函数。不过catch方法还有一个作用，就是在执行resolve回调函数时，如果出现错误，抛出异常，不会停止运行，而是进入catch方法中。

```

p.then((data) => {
    console.log('resolved',data);
},(err) => {
    console.log('rejected',err);
}
);
p.then((data) => {
    console.log('resolved',data);
}).catch((err) => {
    console.log('rejected',err);
});

```

all()

- 返回一个新的 promise, 只有所有的 promise 都成功才成功。

```

javascript
let promise1 = new Promise((resolve,reject)=>{
    setTimeout(()=>{
        resolve(1);
    },2000)
});
let promise2 = new Promise((resolve,reject)=>{
    setTimeout(()=>{
        resolve(2);
    },1000)
});
let promise3 = new Promise((resolve,reject)=>{
    setTimeout(()=>{
        resolve(3);
    },3000)
});
Promise.all([promise1,promise2,promise3]).then(res=>{
    console.log(res);
    //结果为: [1,2,3]
})

```

race()

- 返回一个新的 promise, 第一个完成的 promise 的结果状态就是最终的结果状态(并非是数组中的第一个, 而是第一个完成的 promise)

```
let promise1 = new Promise((resolve, reject) => {
  setTimeout(() => {
    reject(1);
  }, 2000)
});
let promise2 = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve(2);
  }, 1000)
});
let promise3 = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve(3);
  }, 3000)
});
Promise.race([promise1, promise2, promise3]).then(res => {
  console.log(res);
  // 结果: 2
}, rej => {
  console.log(rej);
})
```

finally()

- finally 方法用于指定不管 Promise 对象最后状态如何, 都会执行的操作。

```
promise
  .then(result => {...})
  .catch(error => {...})
  .finally(() => {...});
```

上面代码中, 不管 promise 最后的状态, 在执行完 then 或 catch 指定的回调函数以后, 都会执行 finally 方法指定的回调函数。

async/await 对比 Promise 的优势

- 代码读起来更加同步, Promise 虽然摆脱了回调地狱, 但是 then 的链式调用也会带来额外的阅读负担
- Promise 传递中间值非常麻烦, 而 async/await 几乎是同步的写法, 非常优雅
- 错误处理友好, async/await 可以用成熟的 try/catch, Promise 的错误捕获非常冗余
- 调试友好, Promise 的调试很差, 由于没有代码块, 你不能在一个返回表达式的箭头函数中设置断点, 如果你在一个 then 代码块中使用调试器的步进 (step-over) 功能, 调试器并不会进入后续的 then 代码块, 因为调试器只能跟踪同步代码的每一步。

面向对象程序设计

创建对象

一般使用字面量的形式直接创建对象，但是这种创建方式对于创建大量相似对象的时候，会产生大量的重复代码。但 js 和一般的面向对象的语言不同，在 ES6 之前它没有类的概念。但是可以使用函数来进行模拟，从而产生出可复用的对象创建方式，常见的有以下几种：

工厂模式

- 工厂模式的主要工作原理是用函数来封装创建对象的细节，从而通过调用函数来达到复用的目的。但是它有一个很大的问题就是创建出来的对象无法和某个类型联系起来，它只是简单的封装了复用代码，而没有建立起对象和类型间的关系。

构造函数模式

- js 中每一个函数都可以作为构造函数，只要一个函数是通过 new 来调用的，那么就可以把它称为构造函数。执行构造函数首先会创建一个对象，然后将对象的原型指向构造函数的 prototype 属性，然后将执行上下文中的 this 指向这个对象，最后再执行整个函数，如果返回值不是对象，则返回新建的对象。因为 this 的值指向了新建的对象，因此可以使用 this 给对象赋值。构造函数模式相对于工厂模式的优点是，所创建的对象和构造函数建立起了联系，因此可以通过原型来识别对象的类型。但是构造函数存在一个缺点就是，造成了不必要的函数对象的创建，因为在 js 中函数也是一个对象，因此如果对象属性中如果包含函数的话，那么每次都会新建一个函数对象，浪费了不必要的内存空间，因为函数是所有的实例都可以通用的。

原型模式

- 第三种模式是原型模式，因为每一个函数都有一个 prototype 属性，这个属性是一个对象，它包含了通过构造函数创建的所有实例都能共享的属性和方法。因此可以使用原型对象来添加公用属性和方法，从而实现代码的复用。这种方式相对于构造函数模式来说，解决了函数对象的复用问题。但是这种模式也存在一些问题，一个是没有办法通过传入参数来初始化值，另一个是如果存在一个引用类型如 Array 这样的值，那么所有的实例将共享一个对象，一个实例对引用类型值的改变会影响所有的实例。

组合使用构造函数模式和原型模式

- 第四种模式是组合使用构造函数模式和原型模式，这是创建自定义类型的最常见方式。因为构造函数模式和原型模式分开使用都存在一些问题，因此可以组合使用这两种模式，通过构造函数来初始化对象的属性，通过原型对象来实现函数方法的复用。这种方法很好的解决了两种模式单独使用时的缺点，但是有一点不足的就是，因为使用了两种不同的模式，所以对于代码的封装性不够好。

动态原型模式

- 第五种模式是动态原型模式，这一种模式将原型方法赋值的创建过程移动到了构造函数的内部，通过对属性是否存在的判断，可以实现仅在第一次调用函数时对原型对象赋值一次的效果。这一种方式很好地对上面的混合模式进行了封装。

寄生构造函数模式

- 第六种模式是寄生构造函数模式，这一种模式和工厂模式的实现基本相同，我对这个模式的理解是，它主要是基于一个已有的类型，在实例化时对实例化的对象进行扩展。这样既不用修改原来的构造函数，也达到了扩展对象的目的。它的一个缺点和工厂模式一样，无法实现对象的识别。

继承

- 继承这块知识点算是JavaScript中比较重要且比较难理解的知识，需要读者仔细学习

原型链继承

- 简单理解就是将父类的实例作为子类的原型

```
function Parent() {
  this.isShow = true
  this.info = {
    name: "yhd",
    age: 18,
  };
}

Parent.prototype.getInfo = function() {
  console.log(this.info);
  console.log(this.isShow); // true
}

function Child() {};
Child.prototype = new Parent();

let Child1 = new Child();
Child1.info.gender = "男";
Child1.getInfo(); // {name: "yhd", age: 18, gender: "男"}

let child2 = new Child();
child2.getInfo(); // {name: "yhd", age: 18, gender: "男"}
child2.isShow = false

console.log(child2.isShow); // false
```

- 优点：父类方法可以复用。
- 缺点：父类的所有引用属性会被所有子类共享，更改一个子类的引用属性，其他子类也会受影响；子类型实例不能给父类型构造函数传参。

借用构造函数

- 在子类构造函数中调用父类构造函数，可以在子类构造函数中使用call()和apply()方法

```
function Parent() {
  this.info = {
    name: "yhd",
    age: 19,
  }
}

function Child() {
  Parent.call(this)
}

let child1 = new Child();
child1.info.gender = "男";
console.log(child1.info); // {name: "yhd", age: 19, gender: "男"};

let child2 = new Child();
console.log(child2.info); // {name: "yhd", age: 19}
```

通过使用call()或apply()方法，Parent构造函数在为Child的实例创建的新对象的上下文执行了，就相当于新的Child实例对象上运行了Parent()函数中的所有初始化代码，结果就是每个实例都有自己的info属性。

1、传递参数

相比于原型链继承，盗用构造函数的一个优点在于可以在子类构造函数中像父类构造函数传递参数。

```
function Parent(name) {
    this.info = { name: name };
}
function Child(name) {
    //继承自Parent，并传参
    Parent.call(this, name);

    //实例属性
    this.age = 18
}

let child1 = new Child("yhd");
console.log(child1.info.name); // "yhd"
console.log(child1.age); // 18

let child2 = new Child("wxb");
console.log(child2.info.name); // "wxb"
console.log(child2.age); // 18
```

在上面例子中，Parent构造函数接收一个name参数，并将他赋值给一个属性，在Child构造函数中调用Parent构造函数时传入这个参数，实际上会在Child实例上定义name属性。为确保Parent构造函数不会覆盖Child定义的属性，可以在调用父类构造函数之后再给子类实例添加额外的属性。

- 优点:可以在子类构造函数中向父类传参数；父类的引用属性不会被共享。
- 缺点：子类不能访问父类原型上定义的方法（即不能访问Parent.prototype上定义的方法），因此所有方法属性都写在构造函数中，每次创建实例都会初始化。

组合继承

- 组合继承综合了原型链继承和盗用构造函数继承(构造函数继承)，将两者的优点结合了起来，基本的思路就是使用原型链继承原型上的属性和方法，而通过构造函数继承实例属性，这样既可以把方法定义在原型上以实现重用，又可以让每个实例都有自己的属性。

```
function Parent(name) {
    this.name = name
    this.colors = ["red", "blue", "yellow"]
}
Parent.prototype.sayName = function () {
    console.log(this.name);
}

function Child(name, age) {
    // 继承父类属性
    Parent.call(this, name)
    this.age = age;
}
// 继承父类方法
Child.prototype = new Parent();
```

```

Child.prototype.sayAge = function () {
    console.log(this.age);
}

let child1 = new Child("yhd", 19);
child1.colors.push("pink");
console.log(child1.colors); // ["red", "blue", "yellow", "pink"]
child1.sayAge(); // 19
child1.sayName(); // "yhd"

let child2 = new Child("wxb", 30);
console.log(child2.colors); // ["red", "blue", "yellow"]
child2.sayAge(); // 30
child2.sayName(); // "wxb"

```

上面例子中，Parent构造函数定义了name，colors两个属性，接着又在他的原型上添加了一个sayName()方法。Child构造函数内部调用了Parent构造函数，同时传入了name参数，同时Child.prototype也被赋值为Parent实例，然后又在他的原型上添加了一个sayAge()方法。这样就可以创建child1，child2两个实例，让这两个实例都有自己的属性，包括colors，同时还共享了父类的sayName方法。

- 优点：父类的方法可以复用；可以在Child构造函数中向Parent构造函数中传参；父类构造函数中的引用属性不会被共享。

原型式继承

- 对参数对象的一种浅复制

```

function objectCopy(obj) {
    function Fun() { };
    Fun.prototype = obj;
    return new Fun()
}

let person = {
    name: "yhd",
    age: 18,
    friends: ["jack", "tom", "rose"],
    sayName: function() {
        console.log(this.name);
    }
}

let person1 = objectCopy(person);
person1.name = "wxb";
person1.friends.push("lily");
person1.sayName(); // wxb

let person2 = objectCopy(person);
person2.name = "gsr";
person2.friends.push("kobe");
person2.sayName(); // "gsr"

console.log(person.friends); // ["jack", "tom", "rose", "lily", "kobe"]

```

- 优点：父类方法可复用。
- 缺点：父类的引用会被所有子类所共享；子类实例不能向父类传参。

寄生式继承

- 使用原型式继承对一个目标对象进行浅复制，增强这个浅复制的能力。

```
function objectCopy(obj) {
  function Fun() { };
  Fun.prototype = obj;
  return new Fun();
}

function createAnother(original) {
  let clone = objectCopy(original);
  clone.getName = function () {
    console.log(this.name);
  };
  return clone;
}

let person = {
  name: "yhd",
  friends: ["rose", "tom", "jack"]
}

let person1 = createAnother(person);
person1.friends.push("lily");
console.log(person1.friends);
person1.getName(); // yhd

let person2 = createAnother(person);
console.log(person2.friends); // ["rose", "tom", "jack", "lily"]
```

寄生式组合继承

```
function objectCopy(obj) {
  function Fun() { };
  Fun.prototype = obj;
  return new Fun();
}

function inheritPrototype(child, parent) {
  let prototype = objectCopy(parent.prototype); // 创建对象
  prototype.constructor = child; // 增强对象
  Child.prototype = prototype; // 赋值对象
}

function Parent(name) {
  this.name = name;
  this.friends = ["rose", "lily", "tom"]
}

Parent.prototype.sayName = function () {
  console.log(this.name);
}
```

```
function Child(name, age) {
  Parent.call(this, name);
  this.age = age;
}

inheritPrototype(Child, Parent);
Child.prototype.sayAge = function () {
  console.log(this.age);
}

let child1 = new Child("yhd", 23);
child1.sayAge(); // 23
child1.sayName(); // yhd
child1.friends.push("jack");
console.log(child1.friends); // ["rose", "lily", "tom", "jack"]

let child2 = new Child("yl", 22);
child2.sayAge(); // 22
child2.sayName(); // yl
console.log(child2.friends); // ["rose", "lily", "tom"]
```

- 优点：只调用一次父类构造函数;Child可以向Parent传参;父类方法可以复用;父类的引用属性不会被共享。
- 寄生式组合继承可以算是引用类型继承的最佳模式

垃圾回收机制

标记清除

- JavaScript 中最常用的垃圾收集方式是标记清除(mark-and-sweep)。当变量进入环境（例如，在函数中声明一个变量）时，就将这个变量标记为“进入环境”。从逻辑上讲，永远不能释放进入环境的变量所占用的内存，因为只要执行流进入相应的环境，就可能会用到它们。而当变量离开环境时，则将其标记为“离开环境”。

引用计数

- 另外一种垃圾回收机制就是引用计数，这个用的相对较少。引用计数就是跟踪记录每个值被引用的次数。当声明了一个变量并将一个引用类型赋值给该变量时，则这个值的引用次数就是1。相反，如果包含对这个值引用的变量又取得了另外一个值，则这个值的引用次数就减1。当这个引用次数变为0时，说明这个变量已经没有价值，因此，在在机回收期下次再运行时，这个变量所占有的内存空间就会被释放出来。
- 这种方式引起循环引用的问题：例如：obj1和obj2通过属性进行相互引用，两个对象的引用次数都是2。当使用循环计数时，由于函数执行完后，两个对象都离开作用域，函数执行结束，obj1和obj2还将会继续存在，因此它们的引用次数永远不会是0，就会引起循环引用。

```
function fun() {
  let obj1 = {};
  let obj2 = {};
  obj1.a = obj2; // obj1 引用 obj2
  obj2.a = obj1; // obj2 引用 obj1
}
```

这种情况下，就要手动释放变量占用的内存：


```
obj1.a = null
obj2.a = null
```

哪些情况会导致内存泄漏

以下四种情况会造成内存的泄漏：

- 意外的全局变量：由于使用未声明的变量，而意外的创建了一个全局变量，而使这个变量一直留在内存中无法被回收。
- 被遗忘的计时器或回调函数：设置了 `setInterval` 定时器，而忘记取消它，如果循环函数有对外部变量的引用的话，那么这个变量会被一直留在内存中，而无法被回收。
- 脱离 DOM 的引用：获取一个 DOM 元素的引用，而后面这个元素被删除，由于一直保留了对这个元素的引用，所以它也无法被回收。
- 闭包：不合理的使用闭包，从而导致某些变量一直被留在内存当中。

事件流

- DOM (文档对象模型)结构是一个树型结构，当一个 HTML 元素产生一个事件时，该事件会在元素节点与根结点之间的路径传播，路径所经过的结点都会收到该事件，这个传播过程可称为 DOM 事件流。
- 传播按顺序分为三个阶段：捕获阶段、目标阶段、冒泡阶段

传播过程

捕获阶段

- 事件的处理将从 DOM 层次的根开始，而不是从触发事件的目标元素开始，事件被从目标元素的所有祖先元素依次往下传递。在这个过程中，事件会被从文档根到事件目标元素之间各个继承派生的元素所捕获，如果事件监听器在被注册时设置了 `useCapture` 属性为 `true` (默认为 `false` 值): `element.addEventListener(eventType, fn, useCapture)`那么它们可以被分派给这期间的任何元素以对事件做出处理；否则，事件会被接着传递给派生元素路径上的下一元素，直至目标元素。

目标阶段

- 事件到达目标元素后，进入当前目标阶段，执行函数代码，之后它会接着通过 DOM 节点再进行冒泡。

冒泡阶段

- 当事件在某一 DOM 元素被触发时，例如用户在客户名字节点上点击鼠标，事件将跟随着该节点继承自的各个父节点冒泡穿过整个的 DOM 节点层次，直到它遇到依附有该事件类型处理器的节点。在冒泡过程中的任何时候都可以终止事件的冒泡(调用事件的 `stopPropagation` 方法)，如果不停止事件的传播，事件将一直通过 DOM 冒泡直至到达文档根。

说一下图片的懒加载和预加载

- 预加载：提前加载图片，当用户需要查看时可直接从本地缓存中渲染。
- 懒加载：懒加载的主要目的是作为服务器前端的优化，减少请求数或延迟请求数。
- 两种技术的本质：两者的行为是相反的，一个是提前加载，一个是迟缓甚至不加载。懒加载对服务器前端有一定的缓解压力作用，预加载则会增加服务器前端压力。

mouseover 和 mouseenter 的区别

- `mouseover`：当鼠标移入元素或其子元素都会触发事件，所以有一个重复触发，冒泡的过程。对应的移除事件是 `mouseout`

- mouseenter: 当鼠标移除元素本身（不包含元素的子元素）会触发事件，也就是不会冒泡，对应的移除事件是 mouseleave

事件委托

- 不给每个子节点单独设置事件监听器，而是设置在其父节点上，然后利用冒泡原理设置每个子节点。

事件委托的作用

- 只操作了一次 DOM，提高了程序的性能。

为什么要事件委托

- 在 JavaScript 中，添加到页面上的事件处理程序数量将直接关系到页面的整体运行性能，因为需要不断的操作 dom,那么引起浏览器重绘和回流的可能也就越多，页面交互的事件也就变的越长，这也就是为什么要减少 dom 操作的原因。每一个事件处理函数，都是一个对象，那么多一个事件处理函数，内存中就会被多占用一部分空间。如果要用事件委托，就会将所有的操作放到 js 程序里面，只对它的父级(如果只有一个父级)这一个对象进行操作，与 dom 的操作就只需要交互一次，这样就能大大的减少与 dom 的交互次数，提高性能。

事件委派应用

- 这里会出现常见的面试题，读者可跳过前方知识点，直接上题

给 ul 注册点击事件，然后利用事件对象的 target 来找到当前点击的 li，因为点击 li，事件会冒泡到 ul 上，ul 有注册事件，就会触发事件监听器，这里只操作了一次 DOM，提高了程序的性能。

```
<body>
  <ul>
    <li>1</li>
    <li>2</li>
    <li>3</li>
    <li>4</li>
    <li>5</li>
  </ul>
  <script>
    // 事件委托的核心原理：给父节点添加侦听器， 利用事件冒泡影响每一个子节点
    var ul = document.querySelector('ul');
    ul.addEventListener('click', function(e) {
      // alert('点我应有弹框! ');
      // e.target 这个可以得到我们点击的对象
      e.target.style.backgroundColor = 'pink';
    })
  </script>
</body>
```

这里面试官可能会继续出题，如果通过按钮新增一个新的li标签，如何监听事件

事件循环机制/Event Loop

- Event Loop 即事件循环，是指浏览器或 Node 的一种解决 javascript 单线程运行时不会阻塞的一种机制，也就是我们经常使用异步的原理。
- 在 JavaScript 中，任务被分为两种，一种宏任务（MacroTask）也叫 Task，一种叫微任务（MicroTask）。

MacroTask (宏任务)

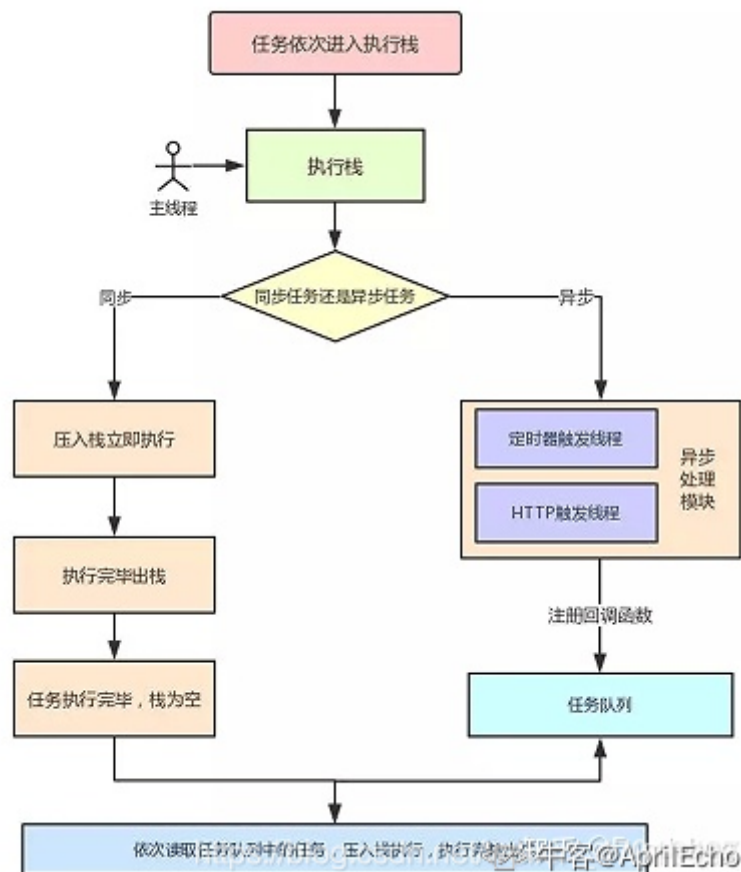
- script 全部代码、setTimeout、setInterval、setImmediate (浏览器暂时不支持, 只有 IE10 支持, 具体可见 MDN)、I/O、UI Rendering。

MicroTask (微任务)

- Process.nextTick (Node 独有)、Promise、Object.observe(废弃)、MutationObserver

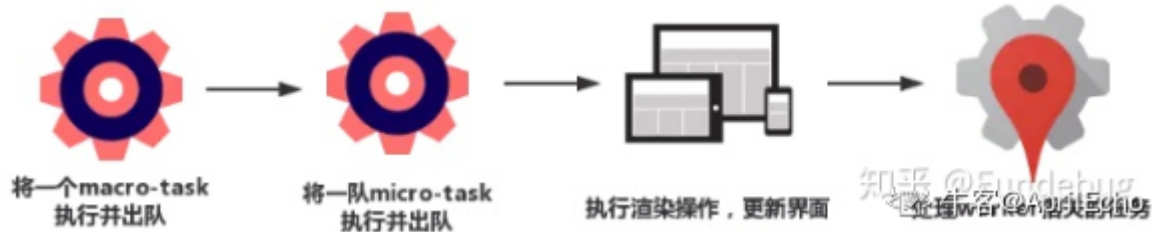
浏览器中的 Event Loop

- Javascript 有一个 main thread 主线程和 call-stack 调用栈(执行栈), 所有的任务都会被放到调用栈等待主线程执行。
- 事件循环中的异步队列有两种: 宏任务队列可以有多个, 微任务队列只有一个。



- 一开始执行栈空,我们可以把执行栈认为是一个存储函数调用的栈结构, 遵循先进后出的原则。micro 队列空, macro 队列里有且只有一个 script 脚本 (整体代码)。全局上下文 (script 标签) 被推入执行栈, 同步代码执行。在执行的过程中, 会判断是同步任务还是异步任务, 通过对一些接口的调用, 可以产生新的 macro-task 与 micro-task, 它们会分别被推入各自的队列里。同步代码执行完了, script 脚本会被移出 macro 队列, 这个过程本质上是队列的 macro-task 的执行和出队的过程。上一步我们出队的是一个 macro-task, 这一步我们处理的是 micro-task。但需要注意的是: 当 macro-task 出队时, 任务是一个一个执行的; 而 micro-task 出队时, 任务是一队一队执行的。因此, 我们处理 micro 队列这一步, 会逐个执行队列中的任务并把它出队, 直到队列被清空。
- 执行渲染操作, 更新界面
- 检查是否存在 Web worker 任务, 如果有, 则对其进行处理
- 上述过程循环往复, 直到两个队列都清空

我们总结一下，每一次循环都是一个这样的过程：



- 当某个宏任务执行完后,会查看是否有微任务队列。如果有，先执行微任务队列中的所有任务，如果没有，会读取宏任务队列中排在最前的任务，执行宏任务的过程中，遇到微任务，依次加入微任务队列。栈空后，再次读取微任务队列里的任务，依次类推。

node 中的事件循环的顺序

- timers 阶段：这个阶段执行 timer (setTimeout、setInterval) 的回调
- I/O callbacks 阶段：处理一些上一轮循环中的少数未执行的 I/O 回调
- idle, prepare 阶段：仅 node 内部使用
- poll 阶段：获取新的 I/O 事件, 适当的条件下 node 将阻塞在这里
- check 阶段：执行 setImmediate() 的回调
- close callbacks 阶段：执行 socket 的 close 事件回调

总结：浏览器环境下，microtask 的任务队列是每个 macrotask 执行完之后执行。而在 Node.js 中，microtask 会在事件循环的各个阶段之间执行，也就是一个阶段执行完毕，就会去执行 microtask 队列的任务。

如题

```
setTimeout(() => {
  console.log('timer1')
  Promise.resolve().then(function () {
    console.log('promise1')
  })
}, 0)
setTimeout(() => {
  console.log('timer2')
  Promise.resolve().then(function () {
    console.log('promise2')
  })
}, 0)
```

- 浏览器端运行结果：timer1=>promise1=>timer2=>promise2
- Node 端运行结果：timer1=>timer2=>promise1=>promise2

微任务和宏任务的区别

- 宏任务：DOM 渲染后触发，如 setTimeout; 是由浏览器规定的
- 微任务：DOM 渲染前触发，如 Promise; 是 ES6 规定的

手撕代码

防抖和节流

防抖实现

所谓防抖，就是指触发事件后 n 秒后才执行函数，如果在 n 秒内又触发了事件，则会重新计算函数执行时间。

```
function debounce(fn, delay) {
  var timeout = null;
  return function (e) {
    clearTimeout(timeout);
    timeout = setTimeout(() => {
      fn.apply(this, arguments);
    }, delay)
  }
}

function handle() {
  console.log('防抖', Math.random())
}

window.addEventListener('scroll', debounce(handle, 50))
```

节流实现

所谓节流，就是指连续触发事件但是在 n 秒中只执行一次函数。节流会稀释函数的执行频率。

```
function throttle(fn, delay) {
  let canRun = true;
  return function () {
    if (!canRun) return;
    canRun = false;
    setTimeout(()=>{
      fn.apply(this,arguments)
      canRun = true
    },delay)
  }
}

function sayHi(e) {
  console.log('节流: ', e.target.innerWidth, e.target.innerHeight);
}

window.addEventListener('resize', throttle(sayHi,500));
```

应用场景

- debounce

1、search 搜索联想，用户在不断输入值时，用防抖来节约请求资源。
2、window 触发 resize 的时候，不断的调整浏览器窗口大小会不断的触发这个事件，用防抖来让其只触发一次

- throttle

1、鼠标不断点击触发，mousedown(单位时间内只触发一次)
2、监听滚动事件，比如是否滑到底部自动加载更多，用 throttle 来判断

深浅拷贝

- 浅拷贝：仅仅是复制了引用，彼此之间的操作会互相影响
- 深拷贝：在堆中重新分配内存，不同的地址，相同的值，互不影响

浅拷贝

```
function shallowCopy(obj) {
  var data={};
  for (var i in obj){
    // for in 循环，也会循环原型链上的属性，所以需要判断一下确定某个对象是否具有带指定名称的属性
    if (obj.hasOwnProperty(i)){
      data[i] = obj[i]
    }
  }
  return data
}
```

深拷贝

```
function deepCopy(obj) {
  if (typeof obj !== 'object') return;
  var newObj = obj instanceof Array ? [] : {};
  for (var key in obj) {
    if (obj.hasOwnProperty(key)) {
      newObj[key] = typeof obj[key] === 'object' ? deepCopy(obj[key]) : obj[key]
    }
  }
  return newObj;
}
```

手动封装Ajax

```
const SERVER_URL = "/server";
let xhr = new XMLHttpRequest();
// 创建 Http 请求
xhr.open("GET", url, true);
// 设置状态监听函数
xhr.onreadystatechange = function() {
  if (this.readyState !== 4) return;
  // 当请求成功时
  if (this.status === 200) {
    handle(this.response);
  } else {
    console.error(this.statusText);
  }
};
// 设置请求失败时的监听函数
xhr.onerror = function() {
  console.error(this.statusText);
};
// 设置请求头信息
xhr.responseType = "json";
xhr.setRequestHeader("Accept", "application/json");
// 发送 Http 请求
xhr.send(null);
```

使用Promise封装AJAX:

```
// promise 封装实现:
function getJSON(url) {
  // 创建一个 promise 对象
  let promise = new Promise(function(resolve, reject) {
    let xhr = new XMLHttpRequest();
    // 新建一个 http 请求
    xhr.open("GET", url, true);
    // 设置状态的监听函数
    xhr.onreadystatechange = function() {
      if (this.readyState !== 4) return;
      // 当请求成功或失败时, 改变 promise 的状态
      if (this.status === 200) {
        resolve(this.response);
      } else {
        reject(new Error(this.statusText));
      }
    };
    // 设置错误监听函数
    xhr.onerror = function() {
      reject(new Error(this.statusText));
    };
    // 设置响应的数据类型
    xhr.responseType = "json";
    // 设置请求头信息
    xhr.setRequestHeader("Accept", "application/json");
    // 发送 http 请求
    xhr.send(null);
  });
  return promise;
}
```

手写 instanceof 方法

- 我们要判断 A 是不是属于 B 这个类型, 只需要当 A 的原型链上存在 B 即可, 即 A 顺着 **proto** 向上查找, 一旦能访问到 B 的原型对象 B.prototype, 表明 A 属于 B 类型, 否则的话 A 顺着 **proto** 最终会指向 null。

```
//方法一
function new_instanceof(left, right) {
  let _left = left.__proto__
  while (_left !== null) {
    if (_left === right.prototype) {
      return true
    }
    _left = _left.__proto__
  }
  return false
}
```

```
//方法二
function myInstanceOf(left, right) {
  // 获取对象的原型
  let proto = Object.getPrototypeOf(left)
  // 获取构造函数的 prototype 对象
  let prototype = right.prototype;
```

```
// 判断构造函数的 prototype 对象是否在对象的原型链上
while (true) {
  if (!proto) return false;
  if (proto === prototype) return true;
  // 如果没有找到, 就继续从其原型上找, Object.getPrototypeOf方法用来获取指定对象的原型
  proto = Object.getPrototypeOf(proto);
}
}
```

call()/apply()/bind()函数

- call、apply、bind 的区别 call 和 apply、bind 都是可以修改 this 的指向, 只是 bind 修改 this 的指向而并不会立即执行, call 和 apply 都会立即执行并修改 this 指向, 只是他们俩的唯一区别是传给函数的参数 call 是 fn.call(ctx, arg1, arg2,.....)的形式, apply是fn.apply(ctx, [arg1,arg2,.....])

手写 call()

```
Function.prototype.myCall = function(context) {
  // 判断调用对象
  if (typeof this !== "function") {
    console.error("type error");
  }
  // 获取参数
  let args = [...arguments].slice(1),
      result = null;
  // 判断 context 是否传入, 如果未传入则设置为 window
  context = context || window;
  // 将调用函数设为对象的方法
  context.fn = this;
  // 调用函数
  result = context.fn(...args);
  // 将属性删除
  delete context.fn;
  return result;
};
```

手写 apply()

```
Function.prototype.myApply=function(context){
  // 获取调用`myApply`的函数本身, 用 this 获取, 如果 context 不存在, 则为 window
  var context = context || window;
  var fn = Symbol();
  context[fn] = this;
  //获取传入的数组参数
  var args = arguments[1];
  if (args == undefined) { //没有传入参数直接执行
    // 执行这个函数
    context[fn]()
  } else {
    // 执行这个函数
    context[fn](...args);
  }
  // 从上下文中删除函数引用
  delete context.fn;
```



```
}
```

手写bind()

```
Function.prototype.myBind = function(context) {  
  // 判断调用对象是否为函数  
  if (typeof this !== "function") {  
    throw new TypeError("Error");  
  }  
  // 获取参数  
  var args = [...arguments].slice(1),  
      fn = this;  
  return function Fn() {  
    // 根据调用方式，传入不同绑定值  
    return fn.apply(  
      this instanceof Fn ? this : context,  
      args.concat(...arguments)  
    );  
  };  
};
```

手写 promise.all()

```
function PromiseAll(promises) {  
  //返回一个 Promise 对象  
  return new Promise((resolve, reject) => {  
    //判断传入的参数是否为数组  
    if (!Array.isArray(promises)) {  
      return reject(new Error("传入的参数不是数组"))  
    }  
    const res = [];  
    //设置一个计时器  
    let count = 0;  
    for (let i = 0; i < promises.length; i++) {  
      Promise.resolve(promises[i]).then(value => {  
        res[i] = value;  
        if (++count === promises.length) {  
          resolve(res)  
        }  
      }).catch(e => reject(e))  
    }  
  })  
}  
  
PromiseAll([1, 2, 3]).then(o => console.log(o))  
PromiseAll([1, Promise.resolve(3)]).then(o=>console.log(o))  
PromiseAll([1, Promise.reject(3).then(o=>console.log(o))])
```

new操作符的实现

new操作符的执行过程：

- (1) 首先创建了一个新的空对象
- (2) 设置原型，将对象的原型设置为函数的 prototype 对象。
- (3) 让函数的 this 指向这个对象，执行构造函数的代码（为这个新对象添加属性）

(4) 判断函数的返回值类型，如果是值类型，返回创建的对象。如果是引用类型，就返回这个引用类型的对象。

具体实现：

```
function objectFactory() {
  let newObject = null;
  let constructor = Array.prototype.shift.call(arguments);
  let result = null;
  // 判断参数是否是一个函数
  if (typeof constructor !== "function") {
    console.error("type error");
    return;
  }
  // 新建一个空对象，对象的原型为构造函数的 prototype 对象
  newObject = Object.create(constructor.prototype);
  // 将 this 指向新建对象，并执行函数
  result = constructor.apply(newObject, arguments);
  // 判断返回对象
  let flag = result && (typeof result === "object" || typeof result ===
"function");
  // 判断返回结果
  return flag ? result : newObject;
}
// 使用方法
objectFactory(构造函数, 初始化参数);
```

JavaScript 两个变量交换值

异或运算

```
var a = 1, // 二进制: 0001
    b = 2; // 二进制: 0010

a = a ^ b; // 计算结果: a = 0011, b = 0010
b = a ^ b; // 计算结果: a = 0011, b = 0001
a = a ^ b; // 计算结果: a = 0010, b = 0001
```

ES6 的解构

```
let a = 1,
    b = 2;

[a, b] = [b, a];
```

利用数组特性进行交换

```
var a = 1,
    b = 2;

a = [a, b];
b = a[0];
a = a[1];
```

斐波那契数列

```
function fibonacci(n) {  
  /*  
    斐波那契：由0和1开始，之后的斐波那契数列每一项都等于前两项之和。  
    斐波那契数列前两项都是1，所以判断n是否等于1或者2，如果是则直接返回1  
    斐波那契数列示例：1、1、2、3、5、8、13、21、34  
  */  
  n = n && parseInt(n);  
  if (n == 1 || n == 2) {  
    return 1;  
  }  
  // 使用arguments.callee实现递归  
  return arguments.callee(n - 2) + arguments.callee(n - 1);  
}  
  
let sum = fibonacci(8)  
console.log(sum) // 21
```

斐波那契数列的第n项

```
// 要求输入一个整数n，请你输出斐波那契数列的第n项（从0开始，第0项为0，第1项是1）。  
function Fibonacci(n) {  
  let arr = [0, 1];  
  for (let i = 2; i <= n; i++) {  
    arr.push(arr[i - 1] + arr[i - 2])  
  }  
  return arr[n]  
}
```

用setTimeout实现setInterval

```
function mySetInterval(fn, millisec) {  
  function interval() {  
    setTimeout(interval, millisec);  
    fn();  
  }  
  setTimeout(interval, millisec)  
}
```

实现函数柯里化

```
function curry(fn) {  
  let judge = (...args) => {  
    if (args.length == fn.length) return fn(...args)  
    return (...arg) => judge(...args, ...arg)  
  }  
  return judge  
}  
  
let curryTest=curry((a,b,c,d)=>a+b+c+d)  
  
console.log(curryTest(1,2,3)(4))
```

```
console.log(curryTest(1,2)(4)(3))
console.log(curryTest(1,2)(3,4))
console.log(curryTest(1)(2)(3)(4))
```

选择排序

```
function selectSort(data) {
  for (var i = 0; i < data.length; i++) {
    var minIndex = i;
    var temp;
    for (var j = i + 1; j < data.length; j++) {
      if (data[j] < data[minIndex]) {
        minIndex = j;
      }
    }
    temp = data[i];
    data[i] = data[minIndex];
    data[minIndex] = temp;
  }
  return data;
}
```

插入排序

```
function insertSort(data) {
  var preIndex, current;
  for (var i = 1; i < data.length; i++) {
    preIndex = i - 1;
    current = data[i];
    while (preIndex >= 0 && current < data[preIndex]) {
      data[preIndex + 1] = data[preIndex];
      preIndex--;
    }
    data[preIndex+1] = current;
  }
  return data;
}
```

快速排序

```
function quickSort(data) {
  if (data.length <= 1) {
    return data;
  }
  var pivot, pivotIndex, left, right;
  left = [];
  right = [];
  pivotIndex = Math.floor(data.length / 2);
  pivot = data.splice(pivotIndex, 1)[0];
  for (var i = 0; i < data.length; i++) {
    if (data[i] < pivot) {
      left.push(data[i])
    } else {
      right.push(data[i])
    }
  }
}
```

```
    }  
  }  
  return quickSort(left).concat([pivot], quickSort(right))  
}
```

冒泡排序

```
function bubbleSort(data) {  
  var temp;  
  for (var i = 0; i < data.length - 1; i++) {  
    for (var j = 0; j < data.length - 1 - i; j++) {  
      if (data[j] > data[j + 1]) {  
        temp = data[j];  
        data[j] = data[j + 1];  
        data[j + 1] = temp;  
      }  
    }  
  }  
  return data;  
}
```

第一秒打印1的问题

```
// 写一个函数，第一秒打印 1，第二秒打印 2  
function f1() {  
  for (let i = 0; i < 5; i++) {  
    setTimeout(() => {  
      console.log(i)  
    }, 1000 * i)  
  }  
}  
  
// f1();  
function f2() {  
  for (var i = 0; i < 5; i++) {  
    (function f(i) {  
      setTimeout(() => {  
        console.log(i)  
      }, 1000 * i)  
    })(i)  
  }  
}  
  
f2();
```

(牛客) 获取 url 中的参数

```
// 获取 url 中的参数  
// 1. 指定参数名称，返回该参数的值 或者 空字符串  
// 2. 不指定参数名称，返回全部的参数对象 或者 {}  
// 3. 如果存在多个同名参数，则返回数组  
// 4. 不支持URLSearchParams方法  
// 输入：  
// http://www.nowcoder.com?key=1&key=2&key=3&test=4#hehe key
```

```
// 输出:
// [1, 2, 3]
function getUrlParam(sUrl, sKey) {

    var param = sUrl.split('#')[0].split('?')[1];
    if (sKey){//指定参数名称
        var str = param.split('&');
        var arr = new Array();//如果存在多个同名参数，则返回数组
        for(var i = 0, len = str.length; i < len; i++){
            var tmp = str[i].split('=');
            if(tmp[0] == sKey){
                arr.push(tmp[1]);
            }
        }
        if (arr.length == 1){//返回该参数的值或者空字符串
            return arr[0];
        } else if (arr.length == 0){
            return "";
        } else {
            return arr;
        }
    } else {//不指定参数名称，返回全部的参数对象 或者 {}
        if(param == undefined || param == ""){
            return {};
        } else {
            var str = param.split('&');
            var arrObj = new Object();
            for(var i = 0, len = str.length; i < len; i++){
                var tmp = str[i].split('=');
                if (!(tmp[0] in arrObj)) {
                    arrObj[tmp[0]] = [];
                }
                arrObj[tmp[0]].push(tmp[1]);
            }
            return arrObj;
        }
    }
}
}
```

青蛙跳台阶

// 一只青蛙一次可以跳上1级台阶，也可以跳上2级。求该青蛙跳上一个n级的台阶总共有多少种跳法（先后次序不同算不同的结果）。

```
function jumpFloor(number) {
    // write code here
    let res = []
    res[0] = 0;
    res[1] = 1;
    res[2] = 2;
    for (let i = 3; i <= number; i++) {
        res[i] = res[i - 1] + res[i - 2]
    }
    return res[number]
}

module.exports = {
    jumpFloor: jumpFloor
}
```

```
};
```

青蛙跳台阶扩展问题

// 一只青蛙一次可以跳上1级台阶，也可以跳上2级.....它也可以跳上n级。求该青蛙跳上一个n级的台阶(n为正整数)总共有多少种跳法。

```
function jumpFloorII(number) {  
  // write code here  
  if (number === 0) return 0  
  return Math.pow(2, number - 1)  
}  
  
module.exports = {  
  jumpFloorII: jumpFloorII  
};
```